# CodeDumper

Turn any codebase into one organized text document.




**CodeDumper** is a command-line tool that scans a project directory, builds a readable directory tree, collects eligible source and text files, and exports everything into a single, structured plain-text file.

Instead of manually opening dozens of files to share a codebase with an AI assistant, researcher, teammate, or reviewer, run one command and generate a project dump.

```
Project Directory
        |
        v
CodeDumper
        |
        +---- Directory Tree
        +---- File Discovery
        +---- Content Collection
        +---- Safety & Exclusion Checks
        |
        v
project-name.txt
```

## Features

- **One-command export:** Generate a complete plain-text snapshot of a project.
- **Readable directory tree:** Understand the structure before reading the source code.
- **Organized file sections:** Each file receives its own `FILE:` section with metadata.
- **Automatic project naming:** Generate `my-project.txt` from the selected project directory.
- **Beginner-friendly menu:** Run `codedumper` with no arguments for a guided menu.
- **Self-update module:** Compare the installed version against official GitHub releases.
- **Smart exclusions:** Skip common virtual environments, dependency directories, build artifacts, caches, and version-control internals.
- **Sensitive-file protection:** Exclude common secrets and credential files by default.
- **Binary-file detection:** Avoid dumping images, executables, and other unsuitable binary content.
- **Large-file protection:** Apply configurable size limits.
- **Custom exclusions:** Skip additional files or directories when needed.

- **Preview mode:** Inspect what would be processed without writing an output file.
- **Deterministic output:** Keep file ordering consistent for easier comparison and review.
- **Cross-platform CLI:** Designed for Windows, Linux, and macOS.
- **Offline operation:** Process local files without requiring an external API or cloud service.

# Installation

## Requirements

- Python 3.10 or later, subject to the version supported by the release.
- A terminal such as PowerShell, Command Prompt, Bash, or Zsh.

## Option 1 — Install from the source repository

Clone the repository and install CodeDumper in editable mode:

```
git clone https://github.com/farhadakhtar/code_dumper.git
cd code_dumper
python -m pip install -e .
```

Verify the installation:

```
codedumper --help
```

> **Windows note:** if `codedumper` is reported as "not recognized", the Python Scripts folder is not on your `PATH`. Either add it, or use the equivalent form (works everywhere, including Linux and macOS with `python3`):
>
> ```
> python -m codedumper --help
> python -m codedumper dump . --dry-run
> ```

## Option 2 — Install a published release

Once CodeDumper is published to PyPI under the package name `codedumper`, users should be able to install it with:

```
python -m pip install codedumper
```

Then launch it from any terminal:

```
codedumper
```

*The PyPI installation command is a planned distribution method; it works only after the package has been published under that name.*

## The CLI experience

Type `codedumper` in any project directory to open the interactive menu. On a true-color terminal (e.g. Windows Terminal) the launch shows the official CodeDumper logo rendered from the bundled `logo.png` artwork — centered, capped at a readable width with margins, aspect-preserved — followed by the installed version and the menu:

```
v1.1.0

 1. Dump the current project
 2. Dump another project
 3. Preview project contents
 4. Update
 5. Configure exclusions
 6. Help and documentation
 7. Exit
```

Terminals without true-color support (classic conhost, pipes, `NO_COLOR`, `dumb` terminals, very narrow windows) skip the artwork and show the same version line and menu instead — never a fake replacement logo.

> Troubleshooting a missing logo: run with `CODEDUMPER_LOGO_DEBUG=1` (e.g. `$env:CODEDUMPER_LOGO_DEBUG=1; codedumper`) to print the exact fallback reason (`not-a-tty`, `no-color`, `dumb-terminal`, `no-truecolor-indicators`, `windows-vt-unavailable`, `narrow-window`, `asset-missing`, `decode-failed`, `encoding`, `write-failed`) to stderr. Note: `$env:` assignments persist for the whole PowerShell session, so remove it afterwards with `Remove-Item Env:\CODEDUMPER_LOGO_DEBUG` — otherwise the diagnostic line keeps appearing on every launch.
> On Windows, an interactive console with virtual-terminal processing renders the logo even when no environment indicator is set; a common remaining cause is a non-UTF-8 console code page — run `chcp 65001` and retry.

- **1. Dump the current project:** scans the current directory and writes `<project-name>.txt` next to the project (outside it, so later scans never re-ingest the dump). You may accept the default path or type another one. A concise summary (discovered / included / excluded / sensitive-skipped counts, duration, output path) is always shown.

- **2. Dump another project:** prompts for a project path, then dumps it the same way.
- **3. Preview project contents:** dry-run scan of the current directory with real counts; nothing is written.
- **4. Update:** regenerates the current project's existing `.txt` dump after explicit confirmation (offers to create one when missing; legacy `.md` dumps are left untouched).
- **5. Configure exclusions:** prompts for extra glob patterns used for dumps in the same session. Custom `--exclude` patterns remain available on direct commands.
- **6. Help and documentation:** prints the full command help.
- **7. Exit.**

Invalid choices re-prompt; `Ctrl+C` exits cleanly; closed input exits without changes.

### Example: dump the current project

```
Select an option [1-7]: 1

Project: my-application
Default output: D:\Projects\my-application.txt
Output path [D:\Projects\my-application.txt] (Enter to accept):

[INFO] Project: my-application
[INFO] Scanning directory...
[INFO] Applying exclusion rules...
[INFO] Collecting eligible files...

Files discovered:       48
Files included:         35
Files excluded:         13
Sensitive files skipped: 2

Output: D:\Projects\my-application.txt

[OK] Export completed successfully.
```

Counts above are illustrative. The actual CLI always displays real scan results and never fabricates statistics.

## Command reference

The CLI supports both interactive use and direct commands. All direct commands below are implemented and covered by automated tests.

### 1. Dump the current directory

```
codedumper
```

When no arguments are supplied, launch the interactive menu. Choose option 1 to dump the current directory.

### 2. Dump a specific project

```
codedumper dump ./my-project
```

Windows example:

```
codedumper dump "D:\Projects\my-project"
```

Without `--output`, the dump is written outside the scanned project: `<project-name>.txt` in the current directory (or in the parent directory when dumping the current directory itself), so later scans never re-ingest the dump. A dump file that ends up inside its own project is always excluded from the scan.

### 3. Choose the output file

```
codedumper dump ./my-project --output ./exports/codebase.txt
```

### 4. Preview without exporting

```
codedumper dump ./my-project --dry-run
```

Show the discovered files, exclusions, and estimated export details without writing the text dump.

### 5. Include hidden files

```
codedumper dump ./my-project --include-hidden
```

Sensitive-file protection must still apply. Hidden files should not automatically bypass security exclusions.

### 6. Exclude additional directories

```

```

```
codedumper dump ./my-project --exclude node_modules --exclude .venv --exclude
dist
```

### 7. Set a maximum file size

```
codedumper dump ./my-project --max-file-size 1MB
```

Supported size units: `B`, `KB`, `MB`, `GB` (case-insensitive, e.g. `500KB`, `2MB`, `1048576`). Invalid values are rejected with an error. `--max-size` works as an alias.

### 8. Choose Markdown output instead

```
codedumper dump ./my-project --format markdown
```

The default format is plain text (`.txt`). `--format markdown` selects the Markdown rendering (`.md`) with language-tagged, dynamically fenced code blocks.

### 9. Display help

```
codedumper --help
```

### 10. Display the installed version

```
codedumper --version
```

The commands and flags above define the intended CLI contract. Implement and test them before documenting them as supported features.

## What does the generated dump look like?

The default output is plain text (`.txt`) — no Markdown headings, no code fences. Source contents are embedded verbatim between `FILE:` and `END FILE:` markers.

Suppose the project has this structure:

```

```

```
my-project/
├── README.md
├── pyproject.toml
├── src/
│   └── app.py
└── config/
    └── settings.yaml
```

CodeDumper generates `my-project.txt`:

```
CODEBASE DUMP: my-project

PROJECT STRUCTURE
my-project/
├── README.md
├── pyproject.toml
├── src/
│   └── app.py
└── config/
    └── settings.yaml

FILE: README.md
Language: Markdown
Size: 48 bytes
Lines: 3

# My Project

Project documentation goes here.

END FILE: README.md

FILE: src/app.py
Language: Python
Size: 45 bytes
Lines: 2

def main():
    print("Hello, world!")

END FILE: src/app.py

EXCLUDED FILES
The following files and paths were discovered during scanning but excluded from
the dump:

.git/ | DEFAULT_NOISE_DIRECTORY | -
```

The example is illustrative. The real output contains the actual eligible file contents, not sample content.

Need Markdown instead? Run with `--format markdown` to get language-tagged, dynamically fenced code blocks (triple backticks are lengthened automatically when source files contain their own backtick sequences). File paths, text encoding, and file boundaries are preserved in both formats so the export remains readable and unambiguous.

## Safety and exclusion policy

A codebase dump can contain confidential information. CodeDumper should use safe defaults.

By default, the tool should:

- Exclude common virtual environments, caches, dependency folders, build outputs, and Git internals.
- Skip common credential files such as `.env`, private keys, and files matching documented secret-file patterns.
- Avoid collecting common secret-bearing configuration files unless explicitly and safely configured.
- Detect binary files and skip them.
- Enforce configurable per-file size limits.
- Avoid following symbolic links by default, preventing unexpected traversal outside the selected project.
- Never upload project contents or transmit them to an external service.
- Report exclusions clearly without printing secret values.

**Important:** Filename exclusions alone cannot guarantee that secrets are removed. Users should review generated dumps before sharing them. CodeDumper should never silently promise that an exported codebase is free of credentials.

## Technical design

CodeDumper should be implemented as a modular Python application.

A suggested structure:

```
CodeDumper/
├── codedumper/
│   ├── __init__.py
│   ├── cli.py
│   ├── scanner.py
│   ├── tree.py
│   ├── exporter.py
│   ├── exclusions.py
│   ├── safety.py
│   └── config.py
├── tests/
│   ├── test_cli.py
│   ├── test_scanner.py
│   ├── test_tree.py
│   ├── test_exporter.py
│   └── test_safety.py
```

```
├── docs/
│   ├── PRD.md
│   ├── ERD.md
│   ├── TRD.md
│   ├── DRD.md
│   └── EDR.md
├── pyproject.toml
├── README.md
├── LICENSE
└── .gitignore
```

The final structure may change after requirements analysis. Do not create unnecessary modules merely to match this example.

### Core responsibilities

| Module | Responsibility |
|---|---|
| `cli.py` | Commands, arguments, interactive menu, and terminal output |
| `scanner.py` | Discover eligible files and directories |
| `tree.py` | Build a readable project tree |
| `renderer.py` | Generate the plain-text (and optional Markdown) document |
| `filter.py` | Apply default and user-defined exclusion rules |
| `classifier.py` | Check file types, size limits, links, and sensitive files |
| `updater.py` | Check official releases and coordinate safe self-updates |

A local file-system CLI does not inherently need a database. Any persistent configuration or other storage must be justified by the requirements.

## Development and testing

Install the project for development:

```
python -m pip install -e .
```

Run the test suite:

```
python -m pytest
```

Test the actual CLI:

```

```

```
codedumper --help
codedumper --version
codedumper dump . --dry-run
codedumper dump . --output ./test-output.txt
```

The project should include tests for:

- Empty directories and nested projects.
- Windows and Unix-style paths.
- Unicode and unusual filenames.
- Binary and oversized files.
- Secret-file exclusions.
- Custom exclusion patterns.
- Permission errors and unreadable files.
- Symbolic links and path traversal.
- Markdown fences inside source files.
- Output creation, overwrite behavior, and invalid arguments.
- Interactive and non-interactive command execution.

Tests must run against temporary test projects and must not expose real credentials or accidentally dump the repository's own secrets.

## Updating CodeDumper

The `codedumper.updater` module compares the installed version against official stable GitHub releases of `farhadakhtar/code_dumper` (prereleases are never offered):

- **Up to date:** it reports that; nothing changes.
- **Update available:** it shows the new version and requires explicit confirmation first. On confirmation it downloads the verified release wheel, installs it with pip, and re-reads the installed version — success is reported only when the versions match. Declining makes no changes.
- **No releases published yet:** automatic updating is honestly reported as not configured, with a safe manual route (`git pull` + reinstall, or install a future release wheel) instead of a fabricated download.

Safety notes: only assets served from the official repository's release downloads are trusted (anything else is refused); editable/source checkouts keep their installation method and always get the manual route; failures (network, permissions, interrupted downloads, install errors) are reported with the manual route and never as success; your projects and dumps are never touched. The core dump workflow has no third-party dependencies and works fully offline — the updater is a separate module.

## Roadmap

### Version 1.1 — Plain-text default, guided menu, update check

- Plain-text (`.txt`) dump format by default; Markdown via `--format`.
- Beginner-friendly interactive menu (spec launch screen).
- Safe self-update check against official GitHub releases.
- Dumps default to outside the scanned project.

### Version 1.0 — Reliable core CLI

- Directory scanning and tree generation.
- Markdown export (plain text became the default in v1.1).
- Interactive terminal menu.
- Non-interactive commands.
- Exclusion and safety rules.
- Automated tests and installation documentation.

### Future releases

- `.codedumperignore` support.
- Multiple export formats.
- Export statistics and token-count estimates.
- Incremental export and content comparison.
- Optional configuration profiles.
- Standalone executables for supported operating systems.

Future features should be implemented only after the core version is reliable.

## Contributing

Contributions are welcome.

1. Open an issue describing the problem or proposed improvement.
2. Keep changes focused and modular.
3. Add automated tests for new behavior.
4. Run the test suite before submitting a pull request.
5. Update documentation when commands or behavior change.

Never include credentials, private keys, or confidential project dumps in issues or pull requests.

## License

CodeDumper is released under the MIT License — see the `LICENSE` file in this repository.

## Project philosophy

CodeDumper should make sharing, reviewing, and understanding codebases easier without turning a simple task into a complicated workflow.

**One command. One organized document. Your codebase, clearly presented.**

```
codedumper
```

Explore your project. Export your code. Understand the whole codebase.